

# Fundamentals to Snowflake

Complete Training for Data Engineers using snowflake

# About the Course

---

- Complete hands-on course
- All topics of Snowflake are covered
- Start from scratch — no prior Snowflake experience needed

What is Snowflake?

# What is Snowflake?

---

- AI Data Cloud Platform — purpose-built for the cloud era
- Not built on legacy database technology
- Cloud native — runs on AWS, Azure, and Google Cloud
- Separates compute from storage for independent scaling
- Delivered as Software as a Service (SaaS)

# Software as a Service (SaaS)

---

## **Quick Setup**

Quick setup with minimal upfront cost. Get started in minutes.

## **Fully Managed**

Fully managed — eliminates the need for customer-maintained infrastructure.

## **Subscription Pricing**

Snowflake uses a subscription-based pricing model — pay for what you use.

Snowflake's SaaS model means no hardware purchases, no software licenses, and no infrastructure management — your team focuses on data, not operations.

# Snowflake Architecture

# Snowflake Architecture Overview

---

## Cloud Services Layer

Authentication, Access Control, Infrastructure Manager, Optimizer, Metadata Manager, Security

## Query Processing Layer

Virtual Warehouses (MPP clusters) — elastic compute, decoupled from storage

## Database Storage Layer

Columnar, compressed storage in cloud — elastic, centralized, cloud-agnostic

# Database Storage Layer

---

- Elastic Storage — decoupled from compute, scales up and down easily
- Snowflake organizes data into optimized, compressed, columnar format
- Data is stored in cloud storage (S3, Azure Blob, GCS)
- Centralized DB storage accessible to all compute nodes
- Data objects are NOT directly accessible by users — access via SQL only

# Query Processing Layer

---

- Elastic Compute — decoupled from storage, scales independently
- All queries are executed in the processing layer
- Snowflake uses 'Virtual Warehouses' (VWs) for query processing
- Each VW is a Massively Parallel Processing (MPP) cluster
- Each VW has its own temporary memory and local cache
- VWs are fully independent — no performance impact between them
- Query result cache is maintained and purged automatically

# Cloud Services **Layer**

---

**Authentication & Access Control:** Verifies user identity and manages access to data

**Infrastructure Manager:** Provisions and manages compute resources for Snowflake

**Query Optimizer:** Determines the most efficient execution plan for every query

**Metadata Manager:** Manages metadata associated with all data stored in Snowflake

**Security:** Encryption, network isolation, and security features to protect data

# Snowflake Editions

# Snowflake Editions

---

## Standard

Base tier — core Snowflake features, ideal for getting started

## Enterprise

All Standard features + multi-cluster warehouses, extended time travel (90 days)

## Business Critical

Enhanced security for sensitive data — HIPAA, PCI DSS compliance

## VPS (Virtual Private Snowflake)

Highest isolation — dedicated hardware, no shared resources

Snowsight UI

# Snowsight

---

Snowsight is Snowflake's modern web-based UI for querying, exploring, and managing data.

## Worksheets

- Write and execute SQL queries
- Syntax highlighting and auto-complete
- Query history and versioning
- Share worksheets with teammates

## Notebooks

- Python and SQL in one place
- Integrated data exploration
- Visualize results inline
- Powered by Snowpark

# Hierarchy of Containers

# Hierarchy of Containers

---

**Organization:** Top-level grouping of Snowflake accounts

**Account:** A single Snowflake instance with its own users, warehouses, and data

**Database:** A logical grouping of schemas — belongs to a single account

**Schema:** A logical grouping of tables, views, and other objects within a database

**Table / View / Stage etc.:** The actual data objects inside a schema

# Database

---

A database is a logical grouping of schemas.  
Each database belongs to a single Snowflake account.

1) `CREATE DATABASE my_database;`

Creates a brand new, empty database inside your Snowflake account. Think of it as setting up a new filing cabinet.

*Real-world: `CREATE DATABASE sales_data;` → a home for all your sales schemas & tables*

2) `USE DATABASE my_database;`

Tells Snowflake which database you're working in for the rest of your session. Like opening a specific cabinet before searching for files.

*Without this, you'd have to write: `my_database.my_schema.my_table` every single time!*

3) `SHOW DATABASES;`

Lists all databases your account has access to, along with details like owner, creation date, and retention settings.

*Great for exploring a new environment — "what databases do we even have?"*

# Schema

---

A schema is a logical grouping of database objects (tables, views, stages, etc.). Each schema belongs to a single database.

1) `CREATE SCHEMA my_schema;`

Creates a schema inside whichever database is active (set by `USE DATABASE`). Best practice is to be explicit about which database it belongs to.

2) `USE SCHEMA my_schema;`

Same pattern as databases — `IF NOT EXISTS` prevents errors on reruns, and a comment helps teammates understand the schema's purpose.

3) `SHOW SCHEMAS IN DATABASE my_database;`

Sets the active schema for your session — after this, you can reference tables by name alone without the full path.

# Data Types

# Snowflake Data Types

---

## Numeric

- NUMBER / DECIMAL
- INT / INTEGER
- FLOAT / DOUBLE

## String / Binary

- VARCHAR / CHAR
- STRING / TEXT
- BINARY / VARBINARY

## Logical

- BOOLEAN

## Date & Time

- DATE
- DATETIME / TIMESTAMP
- TIME

## Semi-Structured

- VARIANT
- OBJECT
- ARRAY

## Geospatial

- GEOGRAPHY
- GEOMETRY

# Snowflake Tables

# Types of Tables

---

## **Permanent Table [Default]**

Default table type. Data persists until explicitly dropped. Supports Time Travel (up to 90 days) and Fail-Safe.

## **Transient Table [Temp/Dev]**

Persists until dropped. No Fail-Safe period. Max 1 day Time Travel. Lower storage cost.

## **Temporary Table [Session]**

Session-scoped. Automatically dropped when session ends. No Fail-Safe.

## **External Table [Read-Only]**

References data in external cloud storage. Read-only. Requires an external stage.

# Snowflake Views

# Snowflake Views

---

A view allows the result of a query to be accessed as if it were a table. The query is specified in the CREATE VIEW statement.

## **Non-Materialized View (Standard)**

Virtual, no data storage. Query re-executed on access. Most flexible.

## **Materialized View**

Behaves like a table — data is persisted. Serverless, faster for repeated queries. Limited use cases.

## **Secure View**

Adds extra layer of security — hides view definition from non-owners. Used for data sharing.

# Materialized Views

---

When to create a Materialized View:

- Query results contain a small number of rows and/or columns relative to base table
- Query results require significant processing — e.g., analysis of semi-structured data
- Query results contain aggregation of large datasets
- The query is on an external table (slower by nature)
- View is queried frequently and the underlying data changes infrequently

# Access Control Framework

# Access Control Framework

---

## **Discretionary Access Control (DAC)**

Each object has an owner, who can in turn grant access to that object.

## **Role-Based Access Control (RBAC)**

Access privileges are assigned to roles, which are in turn assigned to users.

Snowflake combines both DAC and RBAC: ownership + role privilege assignment.

# Access Control — Key Concepts

---

**Securable Object:** An entity to which access can be granted (e.g. table, view, stage, warehouse). Unless explicitly allowed by a grant, access is denied.

**Role:** An entity to which privileges can be granted. Roles are assigned to users. Roles can also be granted to other roles, creating a hierarchy.

**Privilege:** A defined level of access to an object (SELECT, INSERT, USAGE, OWNERSHIP, etc.).

**Grant:** The act of assigning a privilege on an object to a role: `GRANT <privilege> ON <object> TO ROLE <role>`

# Access Control Privileges

---

## Common Table Privileges:

- SELECT
- INSERT
- UPDATE
- DELETE
- TRUNCATE
- REFERENCES
- OWNERSHIP

## Example SQL:

```
GRANT SELECT ON TABLE my_table  
  TO ROLE analyst_role;  
  
GRANT USAGE ON WAREHOUSE compute_wh  
  TO ROLE analyst_role;  
  
GRANT ROLE analyst_role TO USER john_doe;
```

# Types of Roles

---

## Account Roles

Grant privileges on any object in the account. Most common role type.  
Example: SYSADMIN, ANALYST\_ROLE.

## Database Roles

Limit SQL actions to a single database and its objects. Introduced for more granular, database-scoped access control.

## Primary Role

The default assumed role. SET with: `USE ROLE <role_name>`

## Secondary Roles

Additional roles active in a session. Any number can be assigned. Grants combined privileges of all active roles.

# System Defined Roles

---

## **ORGADMIN**

Manages operations at the organization level. Can create accounts and view all accounts/regions.

## **ACCOUNTADMIN**

Top-level account role. Parent to all other roles. Has full visibility.

## **SECURITYADMIN**

Can manage any grant globally. Can create and manage users and roles.

## **SYSADMIN**

Has privileges to create warehouses, databases, and other objects.

## **USERADMIN**

Granted CREATE USER and CREATE ROLE privileges only.

## **PUBLIC**

Pseudo-role automatically granted to all users and roles in the account.

# Snowflake Data Loading

# Stages

---

"Snowflake refers to the location of data files in cloud storage as a stage."

## Internal Stages

- Table Stage — @%<table\_name>
- User Stage — @~
- Named Internal Stage — @<stage\_name> (created explicitly)
- Data stored in Snowflake-managed cloud storage

## External Stages

- Named External Stage — references external S3, Azure Blob, GCS
- Requires cloud credentials / integration
- Data stays in your own cloud storage
- CREATE STAGE command used for both types

# Semi-Structured Data

# Semi-Structured Data

---

"Data that does not conform to traditional structured data standards, but contains tags or markup that identify individual entities."

## Supported Load Formats

- JSON
- Avro
- ORC
- Parquet
- XML

## Supported Unload Formats

- JSON
- Parquet

# Semi-Structured Data Types

---

## **VARIANT (Max 16MB)**

Universal container — stores any semi-structured data type: JSON, Avro, ORC, Parquet, XML

## **ARRAY (Max 16MB)**

Stores an ordered list of values. e.g. ['apple', 'banana', 'cherry']

## **OBJECT (Max 16MB)**

Stores key-value pairs. e.g. {"name": "Alice", "age": 30}

# Unstructured Data

# Unstructured Data

---

"Data that does not fit into a predefined data model or schema." Examples: VCF (genomics), HDF5 (aeronautics), images, audio, video.

- Securely access data files located in cloud storage
- Share file access URLs with collaborators and partners
- Load file access URLs and other file metadata into Snowflake tables
- Process unstructured data using Snowpark or external functions

# Types of URLs to Access Files

---

## Scoped URL

Temporary URL for accessing a specific file in a stage. Expires after a set time.

Function:  
`BUILD_SCOPED_FILE_URL()`

## File URL

Permanent URL that identifies the database, schema, stage, and file path.

Function: `BUILD_STAGE_FILE_URL()`

## Pre-signed URL

Simple HTTPS URL with embedded credentials. Accessible without Snowflake auth.

Function: `GET_PREIGNED_URL()`

# Directory Tables

---

- Stores file-level metadata about files in a stage
- Implicit object layered on top of a stage
- Can be enabled for both external and internal named stages
- Stores URL via BUILD\_STAGE\_FILE\_URL function
- Can be enabled during stage creation or later via ALTER STAGE
- Data can be refreshed manually or automatically (external stages with event notification)

```
CREATE STAGE my_stage DIRECTORY = (ENABLE = TRUE);      ALTER STAGE my_stage REFRESH;
```

# Virtual Warehouses

# Virtual Warehouse

---

A Virtual Warehouse (VW) is a named, Snowflake-managed MPP cluster of compute resources used for query execution.

- Provides the compute resources (CPU, memory, temp storage) for DML and queries
- Does NOT affect data storage — fully decoupled from Database Storage
- Can be started, stopped, resized, and suspended independently
- Each warehouse maintains its own query result cache
- Billed per second of active compute time (minimum 60 seconds)

# Warehouse Sizes

---

Warehouse sizes range from X-Small to 6X-Large. Each size doubles the compute credits consumed per hour.

**X-Small (XS)**

**Small (S)**

**Medium (M)**

**Large (L)**

**X-Large (XL)**

**2X-Large (2XL)**

**3X-Large (3XL)**

**4X-Large (4XL)**

# Standard vs Multi-Cluster Warehouse

---

## Standard Warehouse

- Single cluster of compute resources
- All queries share the same cluster
- Can queue if concurrent queries exceed capacity
- Best for single-user or low-concurrency workloads
- Available on all Snowflake editions

## Multi-Cluster Warehouse

- Multiple clusters of compute resources
- Automatically scales out to handle concurrency
- No queueing — each cluster serves its own queries
- Best for high-concurrency enterprise workloads
- Requires Enterprise Edition or higher

# Scaling Out vs Scaling Up

---

## Scale Out (Multi-Cluster)

Adds more clusters (horizontal scaling). Used for high-concurrency scenarios. Maximum 10 clusters (soft limit). Requires Enterprise Edition.

## Scale Up/Down (Resize)

Changes warehouse size (vertical scaling). XS → S → M → L → XL → 2XL etc. Used when individual queries are slow — need more compute per query.

Rule of thumb: Scale OUT for concurrency (many users), Scale UP for performance (complex queries).

# Scaling Policies

---

## Standard Scaling Policy

Cluster Starts: When a query is queued. Each new cluster waits 20 seconds after the prior one started.

Cluster Shuts: When load can be redistributed from the least-loaded cluster to others.

## Economy Scaling Policy

Cluster Starts: Only when there's enough load to keep a cluster busy for at least 6 minutes.

Cluster Shuts: After 5–6 consecutive checks show it can be consolidated. More cost-efficient, slightly slower to scale.

Snowflake Cortex LLM

# Snowflake Cortex LLM **Function**

---

Snowflake Cortex is a suite of AI features that use large language models (LLMs) to understand unstructured data, answer freeform questions, and provide intelligent assistance.

## **AI SQL Functions**

Task-specific, purpose-built managed functions that automate routine AI tasks directly in SQL. Examples: `COMPLETE()`, `CLASSIFY_TEXT()`, `SENTIMENT()`

## **Helper Functions**

Purpose-built managed functions that reduce failure cases and simplify AI workflows. Examples: `TRY_COMPLETE()`, `EMBED_TEXT_768()`

# Cortex LLM — Available Models

---

Snowflake Cortex provides access to industry-leading LLMs hosted and managed by Snowflake:

**Meta (Llama)**

**Mistral**

**Reka**

**Snowflake Arctic**

**Google (Gemma)**

**Microsoft Phi**

# Continuous Data Protection

# Continuous Data Protection Lifecycle

---

## Current Data Storage

Your live, active data as it exists right now.

## Time Travel

Query, clone, or restore data from any point in the past (up to 90 days for Enterprise).  
Uses AT|BEFORE clause.

## Fail-Safe

7-day disaster recovery period after Time Travel expires. Managed by Snowflake — not accessible via SQL. For catastrophic failure recovery only.

Time Travel Retention: Standard = 1 day (default), Enterprise = up to 90 days

AT (point in time): `SELECT * FROM my_table AT (TIMESTAMP => '2024-01-01 00:00:00');`

BEFORE (offset): `SELECT * FROM my_table BEFORE (STATEMENT => '<query_id>');`

# Snowflake Cloning

# Cloning

---

Cloning creates an exact copy of a database, schema, or table. Uses Zero-Copy Cloning — no data duplication, only metadata is copied. Storage costs only for data that diverges after clone.

- Clonable: Databases, Schemas, Tables, File Formats, Sequences, Streams, Tasks, Pipes, Stages
- NOT clonable: Users, Roles, Grants, Virtual Warehouses (these are account-level, not data objects)
- Clone inherits the structure — no data movement, near-instantaneous
- With Time Travel: can clone an object as it existed at a past point in time

# Cloning — Required Privileges

---

Minimum privileges required to clone objects:

## Tables

Required privilege: SELECT

## Pipes, Streams, Tasks

Required privilege: OWNERSHIP

## All Other Objects

Required privilege: USAGE

CLONE syntax: `CREATE TABLE new_table CLONE source_table;`

With Time Travel: `CREATE TABLE new_table CLONE source_table AT (OFFSET => -3600);`

# Snowflake Marketplace

# Snowflake Marketplace

---

- Provides a wide variety of data, apps, and services from third-party providers
- Discover data/apps/services as per your business needs
- Quick and hassle-free access — no complex ETL pipelines needed
- Native to Snowflake — data is shared directly, not copied
- Many free and freemium offerings available
- Not available for VPS (Virtual Private Snowflake) edition

# Data Exchange

# Data Exchange

---

- Data shared between an invited group of entities (private, controlled sharing)
- Must contact Snowflake Support to enable Data Exchange for your account
- Can share data with partners and external collaborators
- Can share data within your own organization (cross-account sharing)
- Unlike Marketplace (public), Data Exchange is invite-only

# Coterx AI

AI-Powered Data Engineering on Snowflake

# What is Coterx AI?

---

Coterx AI is an intelligent data engineering platform that integrates with Snowflake to accelerate data pipeline development, automate transformations, and provide AI-driven insights for data teams.

- Built natively on Snowflake — leverages Snowpark, Cortex, and Snowflake's security model
- AI-assisted pipeline generation — describe your transformation in plain language
- Automated data quality checks and anomaly detection
- Integrated lineage tracking and impact analysis
- Reduces time-to-insight for data engineering teams

# Coterx AI — Core Capabilities

---

## **AI Pipeline Builder**

Generate Snowflake SQL and Snowpark pipelines from natural language descriptions. Dramatically reduces boilerplate code.

## **Smart Data Profiling**

Automatically profile datasets for quality, completeness, and statistical patterns. Surfaces anomalies proactively.

## **Lineage & Impact Analysis**

Track data lineage end-to-end — understand which tables, views, and pipelines are affected by schema changes.

## **Intelligent Scheduling**

AI-driven job scheduling that optimizes warehouse utilization and minimizes cost while meeting SLAs.

## **Collaboration Hub**

Centralized workspace for data engineers to collaborate on pipelines, review AI suggestions, and manage approvals.

## **Governance Integration**

Integrates with Snowflake's RBAC and data masking policies to ensure data governance compliance.

# Coterx AI Architecture on Snowflake

---

## Coterx AI Application Layer

Pipeline Builder | Data Profiler | Lineage Engine | Scheduler | Collaboration UI

## Snowflake Integration Layer

Snowpark (Python/Scala/Java) | Snowflake Cortex LLM | Snowflake Tasks | Streams

## Snowflake Platform

Virtual Warehouses | Database Storage | RBAC | Data Sharing | Marketplace

## Cloud Infrastructure

AWS | Azure | Google Cloud (Snowflake-managed, cloud-agnostic)

# Coterx AI — Pipeline **Builder**

---

How the AI Pipeline Builder works:

- 1. Describe** Engineer describes the transformation goal in plain English or selects from templates.
- 2. AI Generation** Coterx AI generates optimized Snowpark/SQL code leveraging Snowflake best practices.
- 3. Review & Edit** Engineer reviews the AI-generated code in the built-in IDE with syntax highlighting.
- 4. Test & Validate** Run against sample data — automated test cases generated to validate correctness.
- 5. Deploy** One-click deployment as a Snowflake Task with scheduling and monitoring configured automatically.

# Coterx AI — Data Quality & Profiling

---

## Automated Profiling

- Row counts, null rates, distinct value counts
- Min / max / mean / median statistics
- Schema drift detection across pipeline runs
- Distribution analysis for numeric and categorical fields
- Duplicate detection and referential integrity checks

## Anomaly Detection

- AI-powered outlier detection using statistical models
- Alerts when data distributions shift unexpectedly
- Compare current run vs historical baselines
- Integration with Snowflake Alerts and Notifications
- Configurable thresholds and sensitivity levels

# Coterx AI — Data Lineage & Impact Analysis

---

End-to-end visibility into how data flows through your Snowflake environment:

- Automatically tracks source → transformation → target relationships
- Column-level lineage — trace individual fields through complex transformations
- Impact analysis: 'If I drop this table, what downstream pipelines break?'
- Integrates with Snowflake Access History for audit trail
- Visual lineage graph available in the Coterx UI
- Lineage metadata stored in Snowflake — queryable via SQL

# Coterx AI — Governance & Security

---

## **Snowflake RBAC Integration**

Coterx AI respects Snowflake's role-based access control. Engineers only see and operate on objects they have privileges for.

## **Dynamic Data Masking Support**

Pipelines and profiling respect Snowflake masking policies — sensitive data is never exposed through Coterx.

## **Audit & Compliance**

All pipeline actions are logged. Integrates with Snowflake's `QUERY_HISTORY` and `ACCESS_HISTORY` for compliance reporting.

# Coterx AI — Getting Started

---

Onboarding to Coterx AI as a Data Engineer:

**Step 1: Connect** Connect Coterx AI to your Snowflake account using a dedicated service user with appropriate role grants.

**Step 2: Configure** Set up your warehouse preferences, default schemas, and notification settings in the Coterx console.

**Step 3: Import** Import existing Snowflake objects — Coterx scans your schemas and builds an initial metadata catalog.

**Step 4: Build** Use the Pipeline Builder to create your first AI-assisted pipeline. Start with a simple ingestion or transformation.

**Step 5: Monitor** Set up data quality checks and alerts. Monitor pipeline runs from the Coterx dashboard.

# Coterx AI + Snowflake — Best Practices

---

## Pipeline Design

- Use dedicated warehouses per pipeline type (ingest, transform, report)
- Leverage Snowflake Tasks for orchestration — Coterx handles scheduling
- Prefer incremental loads using Streams over full refreshes
- Tag pipelines with cost center metadata for chargeback

## Performance & Cost

- Let Coterx AI recommend warehouse sizes based on query history
- Enable auto-suspend on all warehouses (default: 5 mins)
- Use result caching for exploratory queries — avoid redundant compute
- Profile data before transforming — catch quality issues early

# Coterx AI — Use Cases for Data Engineers

---

## Automated Ingestion

Generate Snowpipe or COPY INTO pipelines from source specs. Auto-configure file formats and error handling.

## ELT Transformation

AI-generate dbt-style transformations in Snowpark Python or SQL. Validate business logic with AI-generated test cases.

## Data Vault Automation

Generate Data Vault 2.0 structures (Hubs, Links, Satellites) from source schemas automatically.

## Reverse ETL

Push Snowflake data back to operational systems with Coterx's reverse pipeline builder.

# Summary & Next Steps

---

You are now equipped with:

- Deep understanding of Snowflake architecture, editions, and core features
- Knowledge of data loading, semi-structured data, and unstructured data handling
- Mastery of access control, warehouses, time travel, cloning, and data sharing
- Introduction to Snowflake Cortex LLM for AI-powered SQL
- Hands-on knowledge of Coterx AI for AI-assisted data engineering on Snowflake

Next Steps:

- Attempt the SnowPro Associate: Platform practice tests
- Explore Coterx AI's sandbox environment
- Review Snowflake official documentation at [docs.snowflake.com](https://docs.snowflake.com)